إزاي تبني أي أبليكيشن من الصفر — من غير AI

نفس طريقة «سوا» (Flutter + Cubit + Dio)، بس لأي فكرة: مطعم، متجر، عيادة، توصيل، جيم... هتعرف تفكر إزاي، تقسم إيه، وتكتب إيه.

1 الفكرة الكبيرة: حاجات ثابتة وحاجات بتتغير

أي أبليكيشن في الدنيا مقسوم نصين:

النص الثابت (core + shared)	النص اللي بيتغير (features)
الثيم والألوان	HR: حضور، إجازات، مرتبات
التخزين (Cache + Secure)	مطعم: فروع، منيو، سلة، أوردرات
Dio + Interceptor + ApiConstants	عيادة: دكاترة، مواعيد، حجز
رسايل الأخطاء	متجر: أقسام، منتجات، سلة، شحن
AppNavigator	توصيل: مشاوير، خريطة، سواق
الترجمة	
اللغة والثيم (AppSettingsCubit)	
Splash + Login + Register	
Widgets مشتركة (زرار، فيلد، لودينج، خطأ، فاضي)	

💡 يعني: **النص الأول بتعمله مرة واحدة في حياتك** وتنقله من مشروع لمشروع (بتغير الألوان واللينك بس). النص التاني هو اللي بتفكر فيه في كل أبليكيشن جديد. الشرح الكامل للنص الأول بالكود موجود في دليل الـ HR (أقسام 4 لـ 8 و 18 و 19).

2 من الفكرة للكود في 7 خطوات

دي الطريقة اللي تمشي عليها مع أي فكرة. هنطبقها على مطعم:

#	الخطوة	بتسأل نفسك إيه	في المطعم

1	المستخدمين	مين هيفتح الأبليكيشن؟	الزبون. (والمطبخ/الكاشير ليهم لوحة تانية غالباً ويب.)
2	الرحلة	اليوزر بيعمل إيه من أول ما يفتح لحد ما يقفل؟	يفتح ← يختار فرع ← يتفرج على المنيو ← يحط في السلة ← يطلب ← يدفع ← يتابع الأوردر.
3	الأسماء (الكيانات)	طلّع كل «اسم» في الرحلة.	فرع، قسم، صنف، إضافة، سلة، أوردر، عنوان، يوزر.
4	الأفعال	طلّع كل «فعل».	يختار، يضيف، يشيل، يطلب، يدفع، يلغي، يقيّم.
5	الشاشات	كل خطوة في الرحلة = شاشة تقريباً.	Splash، Login، الفروع، المنيو، تفاصيل الصنف، السلة، Checkout، أوردراتي، تتبع الأوردر، حسابي.
6	الـ API	كل شاشة محتاجة أنهي ريكوست؟	`GET /branches`، `GET /menu?branch_id=`، `GET /orders/my`، `POST /orders`...
7	الـ Features	جمّع الشاشات اللي بتتكلم عن نفس الحاجة في فولدر.	`branches`، `menu`، `cart`، `orders`، `profile`، `address`.

💡 قاعدة سهلة: **كل «اسم» = Model، كل «فعل» = دالة في Cubit، كل «مجموعة أسماء قريبة من بعض» = Feature.**

3 الـ Core اللي بتنقله زي ما هو

اعمل لنفسك مشروع «قالب» (Template) فيه الحاجات دي جاهزة، ولما تبدأ أبليكيشن جديد انسخه:

```
my_template/
└── lib/
    ├── main.dart                ← بتغير الاسم والـ home بس
    ├── config/themes/           ← بتغير الألوان بس
    ├── core/
    │   ├── network/             ← بتغير baseUrl + اللينكات
    │   ├── storage/             ← زي ما هو
    │   ├── failure/             ← زي ما هو
    │   └── navigation/          ← زي ما هو
    ├── shared/widgets/          ← زي ما هو
    ├── l10n/                    ← بتزود الكلمات الجديدة
    └── features/
        ├── splash/              ← زي ما هو
        ├── auth/                ← بتغير الحقول حسب السيرفر
        ├── layout/              ← بتغير التابات
        └── settings/            ← زي ما هو
```

وبعد ما تنسخه:

```
# 1) غيّر اسم الباكدج في pubspec.yaml (name: restaurant_app)
# 2) غيّر كل الـ imports القديمة مرة واحدة:
grep -rl "package:my_template/" lib | xargs sed -i ''
's#package:my_template/#package:restaurant_app/#g'
# 3) غيّر الـ Bundle ID / Application ID (أو استخدم باكدج change_app_package_name)
dart run change_app_package_name:main com.yourcompany.restaurant
```

4 قالب أي Feature (انسخ وغيّر الاسم)

ده القالب اللي بيتكرر 90% من الوقت: «هات ليستة من السيرفر واعرضها». غيّر كلمة `Item` بالاسم بتاعك (Branch، Product، Doctor...).

features/items/models/item_model.dart

```dart
class ItemModel {
  final int id;
  final String name;
  final String? image;

  ItemModel({required this.id, required this.name, this.image});

  factory ItemModel.fromJson(Map<String, dynamic> j) => ItemModel(
        id: j['id'] ?? 0,
        name: j['name'] ?? '',
        image: j['image'],
      );
}
```

features/items/cubit/items_cubit.dart

```dart
import 'package:flutter_bloc/flutter_bloc.dart';
import '../../../core/failure/error_message.dart';
import '../../../core/network/dio_helper.dart';
import '../models/item_model.dart';

abstract class ItemsState {}
class ItemsLoading extends ItemsState {}
class ItemsEmpty extends ItemsState {}
class ItemsLoaded extends ItemsState {
  final List<ItemModel> items;
  ItemsLoaded(this.items);
}
class ItemsError extends ItemsState {
  final String message;
  ItemsError(this.message);
}

class ItemsCubit extends Cubit<ItemsState> {
  final String url;
  ItemsCubit(this.url) : super(ItemsLoading());

  Future<void> load({Map<String, dynamic>? query}) async {
    emit(ItemsLoading());
    try {
      final res = await DioHelper.get(url, query: query);
      final list = (res.data['data'] as List)
          .map((e) => ItemModel.fromJson(e))
          .toList();
      emit(list.isEmpty ? ItemsEmpty() : ItemsLoaded(list));
    } catch (e) {
      emit(ItemsError(ErrorMessage.of(e)));
    }
  }
}
```

features/items/presentation/items_screen.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import '../../../core/network/api_constants.dart';
import '../../../shared/widgets/state_views.dart';
import '../cubit/items_cubit.dart';

class ItemsScreen extends StatelessWidget {
  const ItemsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (_) => ItemsCubit(ApiConstants.items)..load(),
      child: Scaffold(
        appBar: AppBar(title: const Text('العنوان')),
        body: BlocBuilder<ItemsCubit, ItemsState>(
          builder: (context, state) {
            final cubit = context.read<ItemsCubit>();
            if (state is ItemsLoading) return const LoadingView();
            if (state is ItemsEmpty) return const EmptyView();
            if (state is ItemsError) {
              return ErrorView(message: state.message, onRetry: cubit.load);
            }
            final items = (state as ItemsLoaded).items;
            return RefreshIndicator(
              onRefresh: cubit.load,
              child: ListView.separated(
                padding: const EdgeInsets.all(16),
                itemCount: items.length,
                separatorBuilder: (_, __) => const SizedBox(height: 10),
                itemBuilder: (_, i) => Card(
                  child: ListTile(
                    title: Text(items[i].name),
                    onTap: () {/* افتح التفاصيل */},
                  ),
                ),
              ),
            );
          },
        ),
      ),
    );
  }
}
```

💡 أي Feature تانية = القالب ده + حاجة جديدة صغيرة (فورم، أو سلة، أو خريطة). متبدأش من

ورقة فاضية أبداً.

5 مثال كامل: أبليكيشن مطعم

الفولدرات

```
lib/features/
├── splash/          (من القالب)
├── auth/            (من القالب — غالباً رقم موبايل + OTP)
├── layout/          تابات: الرئيسية، المنيو، السلة، أوردراتي، حسابي
├── branches/        الفروع + اختيار فرع + أقرب فرع
├── menu/            الأقسام + الأصناف + تفاصيل الصنف والإضافات
├── cart/            السلة (محلية على الموبايل)
├── checkout/        العنوان + طريقة الدفع + تأكيد
├── orders/          أوردراتي + تتبع أوردر
├── address/         عناويني (إضافة/تعديل/حذف)
├── offers/          العروض والكوبونات
└── settings/        (من القالب)
```

اللينكات

core/network/api_constants.dart

```dart
class ApiConstants {
  static const String baseUrl = String.fromEnvironment('BASE_URL',
      defaultValue: 'https://api.example.com/api');

  static const String sendOtp = '/auth/send-otp';
  static const String verifyOtp = '/auth/verify-otp';

  static const String branches = '/branches';
  static const String categories = '/menu/categories';   // ?branch_id=
  static const String products = '/menu/products';       // ?category_id=
  static const String productDetails = '/menu/product';  // ?id=

  static const String addresses = '/addresses';
  static const String validateCoupon = '/coupons/validate';
  static const String createOrder = '/orders';
  static const String myOrders = '/orders/my';
  static const String orderDetails = '/orders/details';  // ?id=
}
```

6 الفروع وأقرب فرع

هي نفس فكرة «مكان الشغل» في البصمة بتاعة HR، بس بالعكس: بدل ما نتأكد إنك جوه الدايرة، بنرتب الفروع من الأقرب للأبعد.

features/branches/models/branch_model.dart

```
class BranchModel {
  final int id;
  final String name;
  final String address;
  final double lat;
  final double lng;
  final bool isOpen;
  final double deliveryFee;
  double? distanceKm; // بنحسبها في الأبليكيشن

  BranchModel({required this.id, required this.name, required this.address,
      required this.lat, required this.lng, required this.isOpen,
      required this.deliveryFee});

  factory BranchModel.fromJson(Map<String, dynamic> j) => BranchModel(
        id: j['id'] ?? 0,
        name: j['name'] ?? '',
        address: j['address'] ?? '',
        lat: (j['lat'] ?? 0).toDouble(),
        lng: (j['lng'] ?? 0).toDouble(),
        isOpen: j['is_open'] ?? false,
        deliveryFee: (j['delivery_fee'] ?? 0).toDouble(),
      );
}
```

features/branches/cubit/branches_cubit.dart

```dart
import 'package:flutter_bloc/flutter_bloc.dart';
import '../../../core/failure/error_message.dart';
import '../../../core/network/api_constants.dart';
import '../../../core/network/dio_helper.dart';
import '../../../core/service/location_service.dart';
import '../../../core/storage/cache_helper.dart';
import '../models/branch_model.dart';

abstract class BranchesState {}
class BranchesLoading extends BranchesState {}
class BranchesLoaded extends BranchesState {
  final List<BranchModel> branches;
  final int? selectedId;
  BranchesLoaded(this.branches, this.selectedId);
}
class BranchesError extends BranchesState {
  final String message;
  BranchesError(this.message);
}

class BranchesCubit extends Cubit<BranchesState> {
  final LocationService location;
  BranchesCubit(this.location) : super(BranchesLoading());

  List<BranchModel> _all = [];

  Future<void> load() async {
    emit(BranchesLoading());
    try {
      final res = await DioHelper.get(ApiConstants.branches);
      _all = (res.data['data'] as List).map((e) => BranchModel.fromJson(e)).toList();

      // لو اليوزر سمح بالموقع: احسب المسافة ورتّب
      final pos = await location.getPosition();
      if (pos != null) {
        for (final b in _all) {
          b.distanceKm = location.distance(pos.latitude, pos.longitude, b.lat, b.lng) /
1000;
        }
        _all.sort((a, b) => a.distanceKm!.compareTo(b.distanceKm!));
      }
      emit(BranchesLoaded(_all, CacheHelper.getData('branch_id')));
    } catch (e) {
      emit(BranchesError(ErrorMessage.of(e)));
    }
  }

  Future<void> select(BranchModel b) async {
    await CacheHelper.saveData('branch_id', b.id); // نفتكره المرة الجاية
```

```
    emit(BranchesLoaded(_all, b.id));
  }
}
```

⚠️ لو المستخدم غيّر الفرع والسلة فيها حاجات من فرع تاني: اسأله «هنفضي السلة، موافق؟».
الأسعار والأصناف ممكن تختلف من فرع لفرع.

7 المنيو (أقسام فوق + أصناف تحت)

features/menu/models/product_model.dart

```
class ProductModel {
  final int id;
  final String name;
  final String? description;
  final String? image;
  final double price;
  final List<AddonModel> addons; // إضافات: جبنة زيادة، صوص...

  ProductModel({required this.id, required this.name, this.description,
      this.image, required this.price, this.addons = const []});

  factory ProductModel.fromJson(Map<String, dynamic> j) => ProductModel(
        id: j['id'] ?? 0,
        name: j['name'] ?? '',
        description: j['description'],
        image: j['image'],
        price: (j['price'] ?? 0).toDouble(),
        addons: ((j['addons'] ?? []) as List).map((e) =>
AddonModel.fromJson(e)).toList(),
      );
}

class AddonModel {
  final int id;
  final String name;
  final double price;
  AddonModel({required this.id, required this.name, required this.price});
  factory AddonModel.fromJson(Map<String, dynamic> j) =>
      AddonModel(id: j['id'] ?? 0, name: j['name'] ?? '', price: (j['price'] ??
0).toDouble());
}
```

الشاشة: `TabBar` فيه الأقسام فوق، وكل تاب جواه القالب بتاع قسم 4 بـ `?category_id=`. أو أسهل: `ChoiceChip` أفقي فوق، ولما تختار قسم تنادي `cubit.load(query: {'category_id':`

`.id})`

```
SizedBox(
  height: 48,
  child: ListView.separated(
    scrollDirection: Axis.horizontal,
    padding: const EdgeInsets.symmetric(horizontal: 16),
    itemCount: categories.length,
    separatorBuilder: (_, __) => const SizedBox(width: 8),
    itemBuilder: (_, i) => ChoiceChip(
      label: Text(categories[i].name),
      selected: categories[i].id == selectedId,
      onSelected: (_) {
        setState(() => selectedId = categories[i].id);
        context.read<ProductsCubit>().load(query: {'category_id': selectedId});
      },
    ),
  ),
)
```

8 السلة (Cart) — الحاجة الجديدة الأهم

السلة مختلفة: **مش بتكلم السيرفر**. هي ليستة على الموبايل لحد ما اليوزر يطلب، ولازم تكون موجودة في **الأبليكيشن كله** (المنيو بيضيف، وأيقونة السلة بتعرض العدد، وشاشة السلة بتعرض الإجمالي) — عشان كده نحطها فوق `MaterialApp` زي `AppSettingsCubit`.

features/cart/models/cart_item.dart

```dart
import '../../menu/models/product_model.dart';

class CartItem {
  final ProductModel product;
  final List<AddonModel> addons;
  final int quantity;
  final String? note; // "من غير بصل"

  const CartItem({required this.product, this.addons = const [],
      this.quantity = 1, this.note});

  double get unitPrice => product.price + addons.fold(0.0, (s, a) => s + a.price);
  double get total => unitPrice * quantity;

  /// نفس الصنف بنفس الإضافات = نفس السطر في السلة
  String get key => '${product.id}-${(addons.map((a) =>
a.id).toList()..sort()).join(',')}';

  CartItem copyWith({int? quantity}) => CartItem(
      product: product, addons: addons, quantity: quantity ?? this.quantity, note: note);

  Map<String, dynamic> toJson() => {
        'product_id': product.id,
        'addon_ids': addons.map((a) => a.id).toList(),
        'quantity': quantity,
        'note': note ?? '',
      };
}
```

features/cart/cubit/cart_cubit.dart

```dart
import 'package:flutter_bloc/flutter_bloc.dart';
import '../models/cart_item.dart';

class CartState {
  final List<CartItem> items;
  final double discount;
  const CartState({this.items = const [], this.discount = 0});

  int get count => items.fold(0, (s, i) => s + i.quantity);
  double get subtotal => items.fold(0.0, (s, i) => s + i.total);
  double get total => (subtotal - discount).clamp(0, double.infinity);
  bool get isEmpty => items.isEmpty;
}

class CartCubit extends Cubit<CartState> {
  CartCubit() : super(const CartState());

  void add(CartItem item) {
    final list = [...state.items];
    final i = list.indexWhere((e) => e.key == item.key);
    if (i == -1) {
      list.add(item);
    } else {
      list[i] = list[i].copyWith(quantity: list[i].quantity + item.quantity);
    }
    emit(CartState(items: list, discount: state.discount));
  }

  void changeQty(String key, int qty) {
    final list = [...state.items];
    final i = list.indexWhere((e) => e.key == key);
    if (i == -1) return;
    if (qty <= 0) {
      list.removeAt(i);
    } else {
      list[i] = list[i].copyWith(quantity: qty);
    }
    emit(CartState(items: list, discount: state.discount));
  }

  void applyDiscount(double value) =>
      emit(CartState(items: state.items, discount: value));

  void clear() => emit(const CartState());
}
```

main.dart (الجزء المهم)

```
runApp(
  MultiBlocProvider(
    providers: [
      BlocProvider(create: (_) => AppSettingsCubit()),
      BlocProvider(create: (_) => CartCubit()), // متاحة في كل الشاشات
    ],
    child: const MyApp(),
  ),
);

// في أي شاشة:
context.read<CartCubit>().add(CartItem(product: p));

// أيقونة السلة بالعدد:
BlocBuilder<CartCubit, CartState>(
  builder: (_, cart) => Badge(
    isLabelVisible: cart.count > 0,
    label: Text('${cart.count}'),
    child: const Icon(Icons.shopping_cart_outlined),
  ),
)
```

💡 لو عايز السلة تفضل موجودة لو اليوزر قفل الأبليكيشن: احفظ `items.map((e) => e.toJson())` كـ JSON في `CacheHelper` مع كل تغيير، ورجّعها في الـ constructor. أو استخدم `hydrated_bloc` اللي بيعمل ده لوحده.

⚠️ السعر النهائي **السيرفر هو اللي يحسبه** وقت الأوردر. اللي في الموبايل للعرض بس — أي حد ممكن يلعب في الأبليكيشن ويغيّر الأسعار.

9 الأوردر والدفع والمتابعة

features/checkout/cubit/checkout_cubit.dart

```dart
import 'package:flutter_bloc/flutter_bloc.dart';
import '../../../core/failure/error_message.dart';
import '../../../core/network/api_constants.dart';
import '../../../core/network/dio_helper.dart';
import '../../cart/cubit/cart_cubit.dart';

abstract class CheckoutState {}
class CheckoutIdle extends CheckoutState {}
class CheckoutSending extends CheckoutState {}
class CheckoutDone extends CheckoutState {
  final int orderId;
  final String? paymentUrl; // لو دفع أونلاين
  CheckoutDone(this.orderId, this.paymentUrl);
}
class CheckoutError extends CheckoutState {
  final String message;
  CheckoutError(this.message);
}

class CheckoutCubit extends Cubit<CheckoutState> {
  final CartCubit cart;
  CheckoutCubit(this.cart) : super(CheckoutIdle());

  Future<void> placeOrder({required int branchId, required int addressId,
      required String payment /* cash / card */, String? coupon}) async {
    if (state is CheckoutSending) return; // ممنوع أوردر مرتين
    emit(CheckoutSending());
    try {
      final res = await DioHelper.post(ApiConstants.createOrder, data: {
        'branch_id': branchId,
        'address_id': addressId,
        'payment_method': payment,
        'coupon': coupon,
        'items': cart.state.items.map((e) => e.toJson()).toList(),
      });
      cart.clear();
      emit(CheckoutDone(res.data['data']['id'], res.data['data']['payment_url']));
    } catch (e) {
      emit(CheckoutError(ErrorMessage.of(e)));
    }
  }
}
```

الدفع أونلاين

أسهل وأأمن طريقة: **السيرفر** يكلّم بوابة الدفع (Paymob، Fawry، Stripe...) ويرجعلك `payment_url`، وإنت تفتحه في WebView أو المتصفح، ولما يخلص ترجع تسأل السيرفر «الأوردر اتدفع؟».

```
// أبسط حل: افتح لينك الدفع في المتصفح
await launchUrl(Uri.parse(paymentUrl), mode: LaunchMode.externalApplication);
// اعمل refresh لما يرجع للأبليكيشن: رجع للتفاصيل الأوردر وشوف حالة الدفع
```

⚠️ **عمرك** ما تحط مفاتيح بوابة الدفع (Secret Key) جوه الأبليكيشن. ده شغل السيرفر بس.

متابعة الأوردر

حالات الأوردر: `pending → accepted → preparing → on_the_way → delivered` (أو `cancelled`). أسهل طريقة للتحديث:

```
// بسيط ومضمون — (Polling) كل 20 ثانية اسأل السيرفر
Timer? _timer;

void startTracking(int orderId) {
  _timer = Timer.periodic(const Duration(seconds: 20), (_) => loadOrder(orderId));
}

@override
Future<void> close() {
  _timer?.cancel(); // مهم: وقّف التايمر لما الشاشة تتقفل
  return super.close();
}
```

الأحسن بعد كده: (Firebase Messaging) **Push Notifications** — السيرفر يبعتلك إشعار لما الحالة تتغير، أو **WebSocket** لو محتاج لحظي (زي مكان الدليفري على الخريطة).

10 أنواع أبليكيشنات تانية وتقسيمها

كل نوع: الـ Features بتاعته، والحاجة الجديدة اللي هتتعلمها فيه. الباقي كله = القالب بتاع قسم 4.

النوع	الـ Features	الحاجة الجديدة / الصعبة
مطعم / كافيه	branches, menu, cart, checkout, orders, address, offers	السلة، الإضافات، أقرب فرع، تتبع الأوردر.

متجر / سوبرماركت	categories, products, search, cart, wishlist, checkout, orders, address	البحث + الفلاتر + Pagination (تحميل صفحة صفحة)، المفضلة.
عيادة / حجز مواعيد (دكاترة، صالون، ملاعب)	specialties, doctors, doctor_details, booking, my_appointments, reviews	اختيار يوم + ساعة متاحة (Slots)، منع حجز نفس الميعاد مرتين، تذكير قبل الميعاد.
توصيل / مشاوير	map, trip_request, trip_tracking, driver_info, wallet, history	الخريطة، الموقع اللحظي، WebSocket، حساب السعر.
تعليم أونلاين	courses, course_details, lessons, video_player, quizzes, progress, certificates	تشغيل فيديو، حفظ مكان الوقوف، تحميل أوفلاين، حماية المحتوى.
عقارات	listings, filters, listing_details, map_view, favorites, contact_agent	فلاتر كتير (سعر، مساحة، منطقة)، خريطة بالعلامات، معرض صور.
جيم / اشتراكات	plans, subscription, classes, booking, attendance(QR), progress	QR Code للدخول، الاشتراكات والتجديد، الجداول.
شات / سوشيال	chats, chat_room, contacts, posts, comments, notifications	رسايل لحظية (WebSocket / Firestore)، رفع صور، إشعارات.
نظام شركة داخلي (HR، مخازن، مبيعات ميدانية)	HR: attendance, requests, tasks, زي visits, reports	الموقع، الصلاحيات حسب الدور (موظف/مدير)، تقارير PDF.

💡 الصلاحيات حسب الدور (موظف / مدير / أدمن): السيرفر يرجع `role` مع اللوجين، تحفظه، وتعرض التابات والزرايز حسبه: `if (role == 'manager') ...` . بس **السيرفر برضه لازم يمنع** — الإخفاء في الأبليكيشن مش حماية.

11 حاجات بتتكرر في أبليكيشنات كتير

أ) Pagination — تحميل صفحة صفحة (متجر، عقارات، أخبار)

features/products/cubit/products_cubit.dart

```
class ProductsState {
  final List<ProductModel> items;
  final bool loadingMore;
  final bool hasMore;
  final String? error;
  const ProductsState({this.items = const [], this.loadingMore = false,
      this.hasMore = true, this.error});
}

class ProductsCubit extends Cubit<ProductsState> {
  ProductsCubit() : super(const ProductsState());

  int _page = 1;
  static const _perPage = 20;

  Future<void> refresh() async {
    _page = 1;
    emit(const ProductsState());
    await loadMore();
  }

  Future<void> loadMore() async {
    if (state.loadingMore || !state.hasMore) return;
    emit(ProductsState(items: state.items, loadingMore: true, hasMore: true));
    try {
      final res = await DioHelper.get(ApiConstants.products,
          query: {'page': _page, 'per_page': _perPage});
      final newItems = (res.data['data'] as List).map((e) =>
ProductModel.fromJson(e)).toList();
      _page++;
      emit(ProductsState(
        items: [...state.items, ...newItems],
        hasMore: newItems.length == _perPage,
      ));
    } catch (e) {
      emit(ProductsState(items: state.items, hasMore: true, error: ErrorMessage.of(e)));
    }
  }
}

// في الشاشة: لما يقرب من آخر الليستة حمّل الصفحة اللي بعدها
NotificationListener<ScrollNotification>(
  onNotification: (n) {
    if (n.metrics.pixels > n.metrics.maxScrollExtent - 300) {
      context.read<ProductsCubit>().loadMore();
    }
    return false;
  },
```

```
  child: ListView.builder(...),
)
```

ب) البحث — متبعتش ريكوست مع كل حرف

```
Timer? _debounce;

void onSearchChanged(String text) {
  _debounce?.cancel();
  // استنى اليوزر يبطّل كتابة نص ثانية، وبعدين ابحث
  _debounce = Timer(const Duration(milliseconds: 500), () {
    context.read<ProductsCubit>().search(text.trim());
  });
}
```

ج) رفع صورة (بروفايل، شكوى، إعلان)

```
// باكدج image_picker
final picked = await ImagePicker().pickImage(source: ImageSource.gallery, imageQuality:
70);
if (picked == null) return;

final form = FormData.fromMap({
  'image': await MultipartFile.fromFile(picked.path, filename: 'photo.jpg'),
});
await DioHelper.dio.post('/profile/avatar', data: form);
```

د) الصور من النت بكاش

```
// باكدج cached_network_image — بتحفظ الصورة فمتتحملش كل مرة
CachedNetworkImage(
  imageUrl: product.image ?? '',
  fit: BoxFit.cover,
  placeholder: (_, __) => const ColoredBox(color: Color(0xFFEEEEEE)),
  errorWidget: (_, __, ___) => const Icon(Icons.image_not_supported),
)
```

هـ) تسجيل الدخول برقم الموبايل (OTP)

نفس `AuthCubit` بس بخطوتين: `sendOtp(phone)` ← شاشة الكود ← `verifyOtp(phone, code)` ← تحفظ التوكن. زوّد عداد 60 ثانية قبل «ابعت تاني» بـ `Timer.periodic`.

و) شغال من غير نت (Offline)

أبسط حل: آخر داتا رجعت من السيرفر احفظها كـ JSON في `CacheHelper`. لو الريكوست فشل، اعرض المحفوظ مع شريط صغير «إنت أوفلاين». للحاجات الكبيرة استخدم `hive` أو `sqflite`.

12 باكدج لكل احتياج

محتاج إيه	الباكدج
إدارة الحالة	`flutter_bloc`
نتورك	`dio`
تخزين بسيط / سري	`shared_preferences` / `flutter_secure_storage`
داتا كتير على الموبايل	`hive` أو `sqflite`
الموقع	`geolocator`
خريطة	`google_maps_flutter` أو `flutter_map` (مجانية)
صور من النت	`cached_network_image`
اختيار صورة / كاميرا	`image_picker`
إشعارات	`firebase_messaging` + `flutter_local_notifications`
فتح لينك / اتصال / واتساب	`url_launcher`
صفحة ويب جوه الأبليكيشن (دفع)	`webview_flutter`
QR (قراءة / عمل)	`mobile_scanner` / `qr_flutter`
فيديو	`video_player` + `chewie`
لحظي (Real-time)	`web_socket_channel` أو `cloud_firestore`
تقارير الأعطال	`firebase_crashlytics`
تحويل التاريخ والأرقام	`intl`

💡 قبل ما تختار باكدج على **pub.dev** بص على 3 حاجات: عدد الـ Likes، آخر تحديث (أقل من سنة)، وإنها بتدعم Android و iOS. وافتح تاب **Example** — فيه كود شغال جاهز.

13 مفيش باك إند؟ استخدم Firebase

لو عميل صغير ومعندوش سيرفر، Firebase بيديك جاهز:

محتاج	من Firebase
تسجيل دخول (إيميل، موبايل، جوجل)	Firebase Auth
قاعدة بيانات لحظية	Cloud Firestore
رفع صور وملفات	Firebase Storage
إشعارات	Cloud Messaging
كود على السيرفر (دفع مثلاً)	Cloud Functions

الهيكل **نفسه** مبيتغيرش: بدل `DioHelper.get` في الداتا سورس بتكتب `FirebaseFirestore.instance.collection('branches').get()`. الكيوبت والشاشة زي ما هما — وده سبب إننا بنفصل الداتا سورس لوحده.

```
dart pub global activate flutterfire_cli
flutterfire configure   # بيربط المشروع بـ Firebase ويعمل firebase_options.dart
```

⚠️ اكتب **Security Rules** في Firestore من أول يوم. القواعد الافتراضية «اختبار» بتفتح الداتا لأي حد بعد 30 يوم أو بتقفلها خالص.

14 Checklist قبل التسليم (لأي أبليكيشن)

- ✅ كل شاشة فيها: لودينج + خطأ بزرار «حاول تاني» + شكل لو الليستة فاضية.
- ✅ كل زرار بيبعت ريكوست بيتقفل وهو بيحمّل (مفيش طلب مرتين).
- ✅ جرّبت كل شاشة: عربي وإنجليزي، فاتح وغامق.
- ✅ جرّبت من غير نت، ونت ضعيف.

- ✅ جرّبت على موبايل صغير وتابلت (مفيش Overflow).
- ✅ لو التوكن خلص بيرجع للوجين لوحده.
- ✅ مفيش أي مفتاح سري أو باسورد في الكود.
- ✅ الصلاحيات (موقع، كاميرا...) مكتوب سببها بوضوح في `Info.plist`.
- ✅ الأيقونة والسبلاش واسم الأبليكيشن صح.
- ✅ رقم الـ build زاد في `pubspec.yaml`.
- ✅ عملت Build Release وجرّبته على موبايل حقيقي (مش Debug بس).
- ✅ ملف التوقيع `.jks` والباسوردات متخزنين في مكان آمن برّه Git.
- ✅ صفحة سياسة الخصوصية جاهزة (الستور بيطلبها).